It's a tremendous productivity boost to find what you're looking for in the output before you've even read it. In *From Writing Specs to Running Them*, we'll see how to view our spec output in different formats.

Starting with a failing spec, as you've done here, is the first step of the Red/Green/Refactor development practice essential to TDD and BDD.[3] With this workflow, you'll make sure each example catches failing or missing code before you implement the behavior you're testing.

The next step after writing a failing spec is to make it pass. For this example, all you have to do is add the following line at the top of the file:

01-getting-started/03/spec/sandwich_spec.rb
```
Sandwich = Struct.new(:taste, :toppings)
```

Here, you've defined a Sandwich struct with two fields, which is all your specs need in order to pass. Usually, you'd put this kind of implementation logic into a separate file, typically in the lib directory. For this simple example, it's fine to define it directly in the spec file.

Now, when you rerun your specs, they'll pass:

```
$ rspec
.

Finished in 0.00101 seconds (files took 0.08408 seconds to load)
1 example, 0 failures
```

The three methods you've used in your spec—describe, it, and expect—are the core APIs of RSpec. You can go a long way with RSpec using just these pieces without any other embellishment.

That said, we can't resist showing you a few more things.

Sharing Setup (But Not Sandwiches)

As you write more specs, you'll find yourself repeating setup code from example to example. This repetition clutters up your tests and makes changing the setup code harder.

Fortunately, RSpec provides ways to share common setup across several examples. Let's begin by adding a second example after the first it block:

01-getting-started/04/spec/sandwich_spec.rb
```
it 'lets me add toppings' do
  sandwich = Sandwich.new('delicious', [])
```

3. https://webuild.envato.com/blog/making-the-most-of-bdd-part-1/